Lab Experiment – 15

Decision Tree

Aim

To implement a **Decision Tree** algorithm using Python for classification.

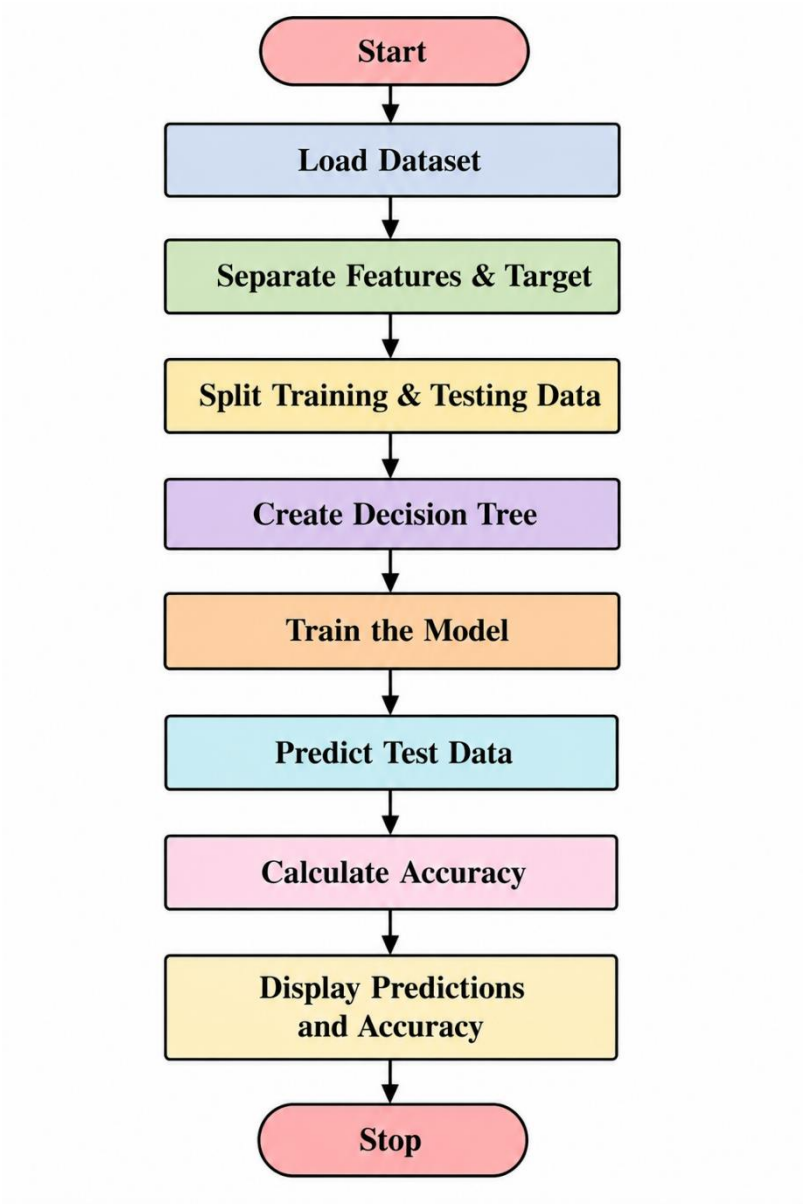
Objective

To construct a decision tree from a dataset and use it to classify new data based on decision rules.

Algorithm

1. Start the program.
2. Load the dataset.
3. Separate the input features and target variable.
4. Split the dataset into training and testing data.
5. Create a Decision Tree classifier.
6. Train the classifier using the training data.
7. Predict the classes for the test data.
8. Calculate the accuracy of the model.
9. Display the predictions and accuracy.
10. Stop.

FlowChart



Code

```
import math
from collections import Counter


# Calculate Entropy
def entropy(data):
    labels = [row[-1] for row in data]
```

```
    count = Counter(labels)

    total = len(labels)
    ent = 0

    for value in count.values():
        probability = value / total
        ent -= probability * math.log2(probability)

    return ent



# Calculate Information Gain
def information_gain(data, index):
    total_entropy = entropy(data)

    values = set(row[index] for row in data)
    weighted_entropy = 0

    for value in values:
        subset = [row for row in data if row[index] == value]

        weighted_entropy += (
            len(subset) / len(data)
        ) * entropy(subset)

    return total_entropy - weighted_entropy
```

```
# Build Decision Tree
def build_tree(data, features):

    labels = [row[-1] for row in data]

    # If all labels are the same
    if len(set(labels)) == 1:
        return labels[0]

    # If no features remain
    if not features:
        return Counter(labels).most_common(1)[0][0]

    # Find feature with maximum information gain
    gains = [information_gain(data, i) for i in features]

    best_index = features[gains.index(max(gains))]

    tree = {best_index: {}}

    # Get possible values
    values = set(row[best_index] for row in data)

    for value in values:

        subset = [
            row for row in data
            if row[best_index] == value
        ]
```

```
            remaining_features = [
                i for i in features
                if i != best_index
            ]

            tree[best_index][value] = build_tree(
                subset,
                remaining_features
            )

    return tree



# Game Dataset
data = [
    ['Sunny', 'Hot', 'No'],
    ['Sunny', 'Cool', 'Yes'],
    ['Rainy', 'Cool', 'Yes'],
    ['Rainy', 'Hot', 'No'],
    ['Overcast', 'Cool', 'Yes'],
    ['Overcast', 'Hot', 'Yes']
]



# Feature names
features = ['Weather', 'Temperature']

# Feature indexes
```

feature_indexes = [0, 1]

Build Decision Tree

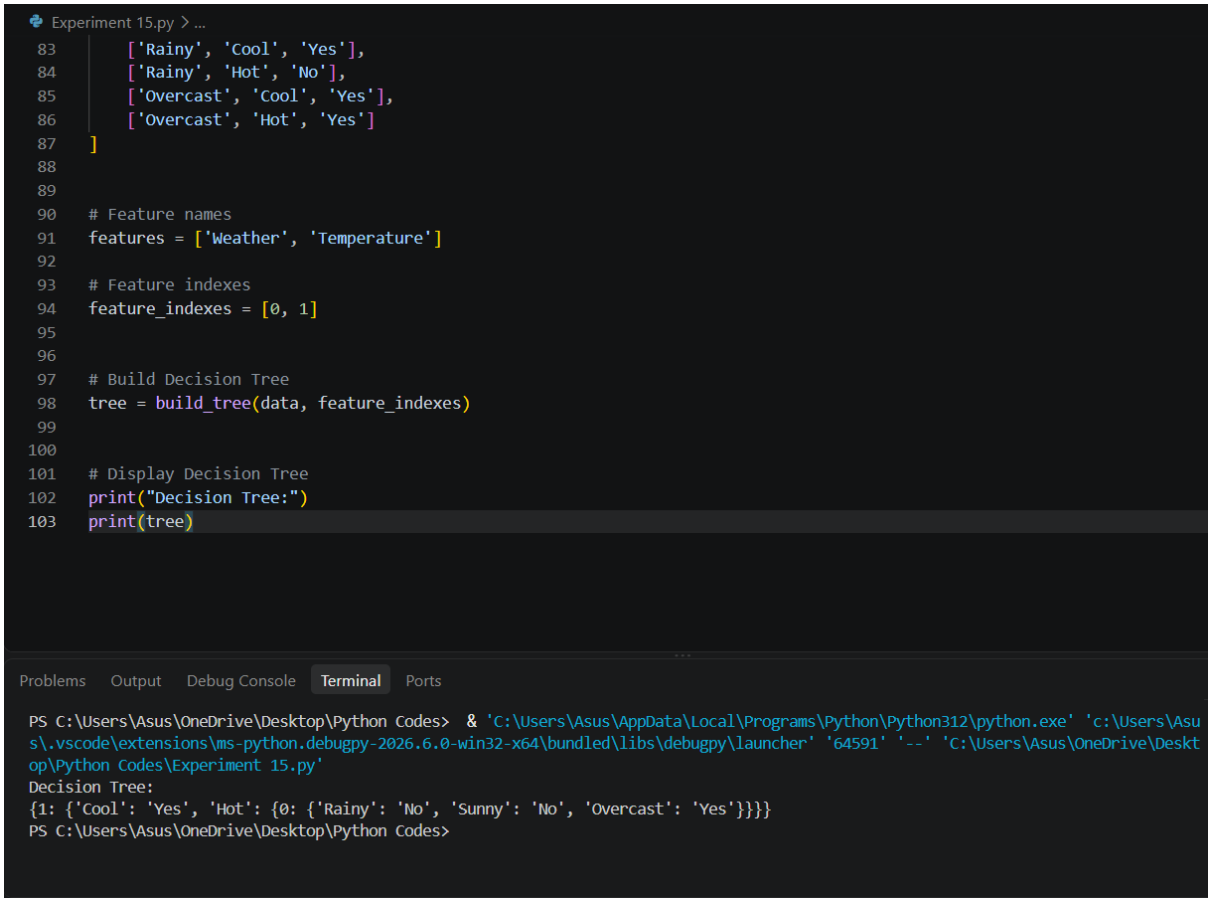
tree = build_tree(data, feature_indexes)

Display Decision Tree

print("Decision Tree:")

print(tree)

OUTPUT:



Result :

The Decision Tree algorithm using ID3 was successfully implemented. The algorithm calculated Entropy and Information Gain and constructed a decision tree from the given game-related dataset.

Conclusion :

The Decision Tree algorithm provides an effective way to represent decisions in a game. By selecting the feature with the highest information gain at each step, the algorithm constructs a tree that can be used to make decisions or classify game situations.